Kubernetes Resource Requests, Limits & QoS

Kubernetes runs multiple applications across a cluster of machines. Each application consumes resources such as CPU and memory.

When several applications share the same infrastructure, Kubernetes needs to solve two important problems:

1. **Scheduling:** Which Node has enough resources to run a particular Pod?
2. **Resource control:** How do we prevent one application from consuming excessive resources and affecting other applications?

Kubernetes addresses these problems using **Resource Requests** and **Resource Limits**.

---

1. Understanding Resource Requests

A Resource Request specifies the amount of CPU or memory that Kubernetes should account for when scheduling a Pod.

Consider a Node with the following capacity:

```
Node
------------------
CPU:     4 cores
Memory:  8 GiB
------------------
```

Suppose we want to deploy multiple applications on this Node.

Before scheduling a Pod, Kubernetes needs to determine whether the Node has sufficient available capacity.

This is where Resource Requests become important.

Example

```
resources:
  requests:
    cpu: "500m"
    memory: "256Mi"
```

This configuration declares that the container requires:

- **CPU:** 0.5 CPU
- **Memory:** 256 MiB

The Kubernetes scheduler uses these requests when determining where the Pod should run.

Understanding CPU Units

Kubernetes represents CPU resources using CPU units.

CPU Value	Meaning
`1`	1 CPU
`1000m`	1 CPU
`500m`	0.5 CPU
`250m`	0.25 CPU
`100m`	0.1 CPU

The letter `m` represents **millicpu**, where:

```
1000m = 1 CPU
```

One CPU unit generally represents one CPU core on a physical machine or one vCPU in a virtualized environment.

For example, a Node with four CPU units could theoretically accommodate the following CPU requests, assuming all four CPU units are allocatable:

```
Pod A → 1000m
Pod B →  500m
Pod C → 1500m
Pod D → 1000m
```

```
----------------
Total → 4000m
```

The Scheduler Uses Requests, Not Current CPU Usage

Consider a Node with four CPUs.

Two Pods are already running:

Pod	CPU Request	Actual CPU Usage
Pod A	2 CPU	0.2 CPU
Pod B	1 CPU	0.1 CPU
Total	**3 CPU**	**0.3 CPU**

Now another Pod arrives with the following request:

```
resources:
  requests:
    cpu: "2"
```

Although the existing Pods are currently using only 0.3 CPU, Kubernetes has already accounted for 3 CPUs through resource requests.

The scheduler calculates:

```
Node capacity       = 4 CPU

Existing requests   = 3 CPU
New Pod request     = 2 CPU

Total requested     = 5 CPU
```

Since the total requested CPU exceeds the Node's capacity, the Pod cannot be scheduled on that Node.

It may remain in the `Pending` state.

```
kubectl get pods
```

To investigate the reason:

```
kubectl describe pod <pod-name>
```

An event might report:

```
Insufficient cpu
```

Why doesn't Kubernetes schedule based on current utilization?

CPU consumption changes continuously. An application consuming very little CPU right now might suddenly require much more under heavy traffic.

Scheduling based only on current usage would make resource planning unreliable.

Resource Requests allow Kubernetes to account for the expected resource needs of workloads.

Does a Request Mean Constant Resource Consumption?

No.

Consider:

```
resources:
  requests:
    cpu: "500m"
```

This does not mean the application continuously consumes exactly 500m CPU.

Its actual consumption might vary:

```
Normal traffic    → 100m
Moderate traffic  → 350m
Heavy traffic     → 700m
```

The actual CPU usage depends on application demand and available resources.

A request is primarily a scheduling requirement, not a fixed consumption value or an upper limit.

Key distinction:

- Resource Request: Capacity accounted for during scheduling.
- Actual Usage: Resources the application consumes while running.

---

2. Understanding Resource Limits

A Resource Limit defines the maximum amount of a resource that a container is allowed to consume.

Consider:

```
resources:
  limits:
    cpu: "1"
    memory: "512Mi"
```

This configuration sets two runtime constraints:

- CPU consumption is limited to one CPU unit.
- Memory usage is constrained to 512 MiB.

Resource limits are enforced on the Node through the kubelet, container runtime, and operating system mechanisms.

However, CPU and memory limits behave differently.

2.1 CPU Limits

Consider a container with:

```
resources:
  requests:
    cpu: "250m"
  limits:
    cpu: "500m"
```

The application normally requires 250m CPU.

During high traffic, it attempts to consume 700m CPU.

However, the configured limit is only 500m.

Kubernetes enforces the CPU limit through a mechanism called **CPU throttling**.

```
Application wants 700m CPU
          ↓
CPU limit is 500m
          ↓
CPU usage is throttled
          ↓
Application continues running
```

CPU throttling restricts the amount of CPU time available to the container.

The application is not normally terminated merely because it attempts to use more CPU than its configured limit.

However, excessive CPU throttling can slow down request processing and affect application performance.

2.2 Memory Limits

Memory limits behave differently from CPU limits.

Consider:

```
resources:
  limits:
    memory: "256Mi"
```

Suppose a Spring Boot application's memory usage keeps increasing:

```
150 MiB
   ↓
200 MiB
   ↓
240 MiB
```

```
   ↓
300 MiB
```

Unlike CPU time, memory already allocated to a process cannot simply be reduced through throttling.

If a container exceeds its memory limit, the operating system may terminate a process through an **Out-of-Memory (OOM) kill**.

Kubernetes can report the container's termination reason as:

```
OOMKilled
```

Depending on the restart policy and workload configuration, the container may be restarted.

If the memory problem continues, repeated failures may eventually result in:

```
CrashLoopBackOff
```

Example

```
Application starts
        ↓
Memory usage increases
        ↓
Memory limit exceeded
        ↓
OOMKilled
        ↓
Container restarted
        ↓
Memory limit exceeded again
        ↓
Repeated crashes
        ↓
CrashLoopBackOff
```

Memory-limit enforcement is reactive. Exceeding a memory limit does not necessarily mean the container is terminated at the exact instant the threshold is

crossed.

CPU Limits vs Memory Limits

CPU Limit	Memory Limit
Enforced through CPU throttling	Enforced through memory constraints
Excess demand reduces available CPU time	Excess usage may result in an OOM kill
Container normally continues running	Container may be terminated
Can cause performance degradation	Can cause application crashes and restarts

3. Configuring Requests and Limits Together

Resource Requests and Limits are commonly configured together.

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

How Kubernetes interprets this configuration

Resource	Request	Limit
CPU	250m	500m
Memory	256Mi	512Mi

During scheduling, Kubernetes accounts for:

```
CPU    → 250m
Memory → 256Mi
```

During execution, the configured limits apply:

```
CPU    → 500m
Memory → 512Mi
```

This allows the application to consume more than its requested CPU when sufficient resources are available, without exceeding the configured CPU limit.

Requests help Kubernetes decide where a Pod can run. Limits control resource consumption while the container is running.

---

4. Choosing Appropriate Requests and Limits

Resource configuration affects both application performance and cluster utilization.

4.1 Why Not Always Set Request = Limit?

Consider an API server that normally consumes:

```
100m CPU
```

During a traffic spike, it might need:

```
600m CPU
```

A possible configuration is:

```
resources:
  requests:
    cpu: "100m"
  limits:
    cpu: "1"
```

Kubernetes accounts for 100m CPU during scheduling, but the container can consume more CPU when required and when capacity is available.

This provides flexibility for workloads with changing resource requirements.

4.2 Problems with Extremely Low Requests

Suppose an application normally needs 500m CPU, but its request is configured as:

```
resources:
  requests:
    cpu: "10m"
```

Now consider a Node with four CPUs.

```
Node CPU capacity = 4000m

400 Pods × 10m = 4000m
```

Based only on CPU requests, the scheduler might consider this configuration acceptable.

However, if every Pod actually needs 500m CPU, the Node becomes heavily contended.

Applications compete for available CPU time, potentially causing slow responses and degraded performance.

Requests should reflect realistic resource requirements rather than artificially small values.

4.3 Problems with Extremely High Requests

Consider an application that normally consumes 200m CPU but declares:

```
resources:
  requests:
    cpu: "3"
```

Kubernetes accounts for three CPUs during scheduling, even though the application normally uses much less.

This can lead to:

- Poor resource utilization.
- Unnecessary scheduling restrictions.
- Pods remaining Pending despite low actual CPU usage.

Resource Requests should be selected based on expected workload requirements.

4.4 What Happens When Only a Limit Is Defined?

Consider:

```
resources:
  limits:
    cpu: "500m"
```

No CPU request is explicitly provided.

If no admission mechanism supplies a different default request, Kubernetes uses the CPU limit as the request.

Effectively:

```
CPU Request = 500m
CPU Limit   = 500m
```

The same behavior applies to memory.

This is important because it can also affect the Pod's Quality of Service classification.

---

5. Quality of Service (QoS) in Kubernetes

A Kubernetes Node has finite CPU and memory resources.

When resource consumption becomes too high, the Node may experience resource pressure.

For example:

```
Node Memory: 8 GiB

Pod A
Pod B
Pod C
Pod D
```

If available memory becomes critically low, Kubernetes may need to terminate or evict workloads to protect the Node.

Kubernetes uses **Quality of Service (QoS) classes** to classify Pods based on their resource configurations.

There are three QoS classes:

1. Guaranteed
2. Burstable
3. BestEffort

QoS influences how Kubernetes treats Pods during resource pressure, particularly when making eviction decisions.

---

6. Guaranteed QoS

A Pod qualifies for Guaranteed QoS when every container has both CPU and memory requests and limits configured, and the corresponding request and limit values are equal.

Example

```
resources:
  requests:
    cpu: "500m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
```

Here:

```
CPU Request    = CPU Limit

Memory Request = Memory Limit
```

For a Pod with multiple containers, these requirements must be satisfied by every container.

Characteristics

- CPU and memory requirements are explicitly declared.
- CPU requests and limits are equal.
- Memory requests and limits are equal.
- Provides the strongest QoS-based protection during Node resource pressure.

Guaranteed does not mean the Pod can never be terminated or evicted.

---

7. Burstable QoS

Burstable QoS applies when a Pod has some CPU or memory resource requests or limits configured but does not satisfy the requirements for Guaranteed QoS.

Example

```
resources:
  requests:
    cpu: "200m"
    memory: "256Mi"
  limits:
    cpu: "1"
    memory: "512Mi"
```

Here:

```
CPU Request = 200m
CPU Limit   = 1000m
```

```
Memory Request = 256Mi
Memory Limit   = 512Mi
```

The application has a smaller scheduling requirement than its configured resource limits.

This allows it to use additional resources when needed.

A Pod can also fall into the Burstable category when only some CPU or memory requests and limits are specified.

Characteristics

- Some resource configuration exists.
- Guaranteed QoS conditions are not fully satisfied.
- Resource consumption can exceed requested values.
- QoS protection is generally between Guaranteed and BestEffort.

---

8. BestEffort QoS

BestEffort applies when none of the containers in a Pod has CPU or memory requests or limits configured.

Example

```
containers:
  - name: nginx
    image: nginx:alpine
```

Notice that there is no `resources` section.

The Pod has no explicitly configured CPU or memory requests or limits.

Characteristics

- No CPU requests or limits.
- No memory requests or limits.

- Can consume available resources.
- Receives the weakest QoS-based protection under Node resource pressure.

BestEffort Pods are particularly vulnerable to eviction when the Node experiences resource shortages.

---

9. Comparing Kubernetes QoS Classes

Feature	Guaranteed	Burstable	BestEffort
CPU requests	Required	May be configured	Not configured
Memory requests	Required	May be configured	Not configured
CPU limits	Required	May be configured	Not configured
Memory limits	Required	May be configured	Not configured
Request = Limit	Yes, for CPU and memory	Not required	Not applicable
Relative QoS protection	Highest	Intermediate	Lowest

QoS and Pod Eviction

When a Node experiences severe resource pressure, Kubernetes may evict Pods to reclaim resources.

A simplified order of protection is:

```
Least protected
      |
      v
  BestEffort
      |
      v
   Burstable
      |
      v
  Guaranteed
      |
      v
Most protected
```

However, Kubernetes does not blindly evict Pods in this order.

Actual eviction decisions also depend on factors such as:

- Whether resource usage exceeds requests.
- Pod priority.
- The amount of resource consumption relative to requests.

Therefore, Guaranteed QoS should not be interpreted as absolute protection against eviction or termination.

---

10. Useful kubectl Commands

The `kubectl` command-line tool is used to interact with Kubernetes clusters.

It allows us to inspect workloads, identify failures, and troubleshoot applications.

10.1 kubectl get

The `get` command retrieves information about Kubernetes resources.

List Pods

```
kubectl get pods
```

List Deployments

```
kubectl get deployments
```

List Services

```
kubectl get services
```

List Nodes

```
kubectl get nodes
```

List common resources in the current namespace

```
kubectl get all
```

Note that `kubectl get all` does not literally list every Kubernetes resource type.

Get Additional Pod Information

```
kubectl get pods -o wide
```

Example output:

```
NAME   READY  STATUS   IP           NODE
api    1/1    Running  10.244.0.5   minikube
```

The `-o wide` option provides additional details such as the Pod's IP address and the Node on which it is running.

---

11. kubectl describe

The `describe` command provides detailed information about a Kubernetes resource.

Suppose:

```
kubectl get pods
```

returns:

```
NAME   READY   STATUS
api    0/1     Pending
```

We know the Pod is Pending, but we do not yet know why.

Use:

```
kubectl describe pod api
```

The output can contain:

- Pod metadata
- Assigned Node
- Pod IP
- Container information
- Image details
- Ports
- Resource Requests
- Resource Limits
- Container states
- Restart counts
- Volumes
- Pod conditions
- Events

Why Are Events Important?

The Events section near the bottom of `kubectl describe` often reveals why a Pod cannot start or why it is experiencing problems.

For example:

```
Warning  FailedScheduling

0/1 nodes are available:
1 Insufficient cpu.
```

This indicates that the scheduler could not place the Pod because the available CPU capacity, based on resource requests, was insufficient.

The `describe` command is particularly useful for investigating Pending Pods, scheduling failures, and container startup problems.

---

12. kubectl logs

The `logs` command displays output written by an application running inside a container.

Suppose:

```
kubectl get pods
```

returns:

```
NAME      READY   STATUS
backend   0/1     CrashLoopBackOff
```

The Pod's container is repeatedly failing.

To inspect the application output:

```
kubectl logs backend
```

For example, a Spring Boot application might produce:

```
Connection refused: mysql:3306
```

This suggests that the application cannot establish a connection with MySQL.

Possible causes could include an unavailable database, incorrect connection configuration, or a networking issue.

The logs provide an application-level starting point for further investigation.

For a container that has already restarted, the previous container's logs may also be useful:

```
kubectl logs backend --previous
```

13. Basic Kubernetes Troubleshooting Workflow

When a Kubernetes application is not working correctly, these three commands provide a useful starting point.

```
kubectl get pods
       |
       v
Check the Pod status
       |
       v
kubectl describe pod <pod-name>
       |
       v
Inspect Pod details and events
       |
       v
kubectl logs <pod-name>
       |
       v
Inspect application output
```

Command	Primary Purpose
`kubectl get`	Check resource status
`kubectl describe`	Inspect resource details and events
`kubectl logs`	Investigate application-level output

These commands are frequently used together when debugging scheduling failures, container crashes, configuration problems, and application startup errors.

14. Quick Revision

Resource Requests

Requests specify the amount of resources Kubernetes should account for when scheduling workloads. They do not represent fixed resource consumption.

Resource Limits

Limits define runtime resource boundaries. CPU limits are generally enforced through throttling, while memory limits can lead to OOM termination.

Quality of Service

QoS classifies Pods according to their CPU and memory resource configuration.

```
Guaranteed
  CPU Request = CPU Limit
  Memory Request = Memory Limit
  For every container

Burstable
  Some CPU/Memory configuration
  But Guaranteed conditions not met

BestEffort
  No CPU/Memory requests or limits
```

kubectl Debugging

```
kubectl get pods

kubectl describe pod <pod-name>

kubectl logs <pod-name>
```

Resource Requests help Kubernetes make scheduling decisions, Resource Limits control runtime resource consumption, and QoS classifications influence how Pods are treated during resource pressure.